

Parallel Image Processing Pipeline Using Hybrid MPI and OpenMP on HPC

Dheeraj Vulchi
Texas State University
Computer Science Department
San Marcos, Texas, USA

Swetha Zallipalli
Texas State University
Computer Science Department
San Marcos, Texas, USA

Abstract—Image processing applications are widely used in artificial intelligence, autonomous vehicles, medical imaging, and computer vision systems. Modern high-resolution images contain millions of pixels, making sequential image processing computationally expensive and time consuming. This project presents a parallel image processing pipeline using OpenMP, MPI, and a Hybrid MPI+OpenMP approach on a High Performance Computing (HPC) environment. A 4K image with over 8.3 million pixels was processed using grayscale conversion, brightness enhancement, and threshold filtering. Experimental results show that the hybrid implementation achieved the best performance with a speedup of 3.91× compared to the sequential implementation.

Index Terms—Parallel Computing, MPI, OpenMP, Hybrid Parallelism, HPC, Image Processing

I. INTRODUCTION

Image processing is the method of analyzing or modifying digital images using computational algorithms. It is widely used in artificial intelligence, autonomous vehicles, medical imaging, surveillance systems, and computer vision applications.

Modern high-resolution images such as 4K images contain millions of pixels. Processing these images sequentially requires high computational time and becomes inefficient for large-scale applications. Parallel computing techniques improve performance by distributing computation across multiple processors and threads.

This project compares four implementations:

- Sequential Processing
- OpenMP Processing
- MPI Processing
- Hybrid MPI + OpenMP Processing

II. PROJECT OBJECTIVES

The objectives of this project are:

- Process 4K images efficiently
- Compare sequential and parallel execution models
- Measure speedup and parallel efficiency
- Analyze communication overhead
- Validate correctness of outputs

III. SYSTEM CONFIGURATION

The project was executed on the Lonestar6 HPC cluster at TACC.

A. Hardware Environment

- AMD EPYC 7763 Processors
- Shared and Distributed Memory Architecture
- Multi-core HPC Environment

B. Software Environment

- C++
- OpenMP
- MPI
- GCC Compiler
- TAU Profiler
- mpiP Profiler

IV. IMAGE PROCESSING PIPELINE

A 4K image with the following resolution was processed:

$$3840 \times 2160 = 8.3 \text{ million pixels}$$

The pipeline applied three filters:

A. Grayscale Conversion

$$Gray = 0.299R + 0.587G + 0.114B$$

B. Brightness Enhancement

Brightness was increased by adding 30 and clamping values between 0 and 255.

C. Threshold Filtering

Binary thresholding was applied using:

$$Threshold = 100$$

V. PARALLELIZATION STRATEGIES

A. Sequential Implementation

The sequential implementation uses a single CPU core and serves as the baseline reference.

B. OpenMP Implementation

The OpenMP version uses shared memory parallelism with four threads.

```
#pragma omp parallel for schedule(static)
for (int i = 0; i < img.size(); i++) {
    applyGrayscale(img[i]);
}
```

C. MPI Implementation

MPI divides image rows across multiple processes using:

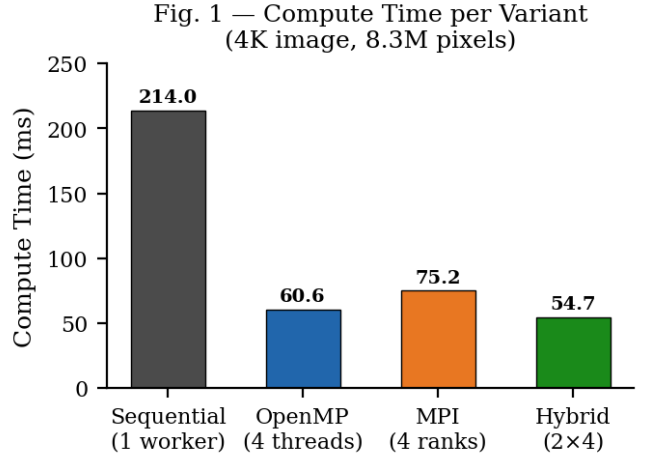
- MPI_Scatterv
- MPI_Gatherv

D. Hybrid MPI + OpenMP

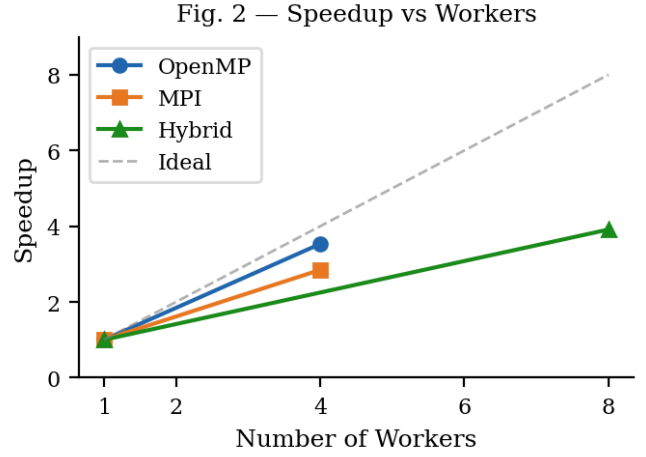
The hybrid implementation combines MPI and OpenMP using:

- 2 MPI processes
- 4 OpenMP threads per process

This achieved the best overall performance.



(a) Compute Time Comparison



(b) Speedup Comparison

Fig. 1: Performance Comparison

VI. PERFORMANCE RESULTS

B. Parallel Efficiency

A. Execution Time Comparison

TABLE I: Execution Time Comparison

Implementation	Time (ms)
Sequential	214
OpenMP	60.6
MPI	75.2
Hybrid	54.7

TABLE II: Parallel Efficiency Analysis

Variant	Workers	Speedup	Efficiency
OpenMP	4	3.53×	88.3%
MPI	4	2.85×	71.1%
Hybrid	8	3.91×	48.9%

Fig. 3 — Parallel Efficiency & Loss

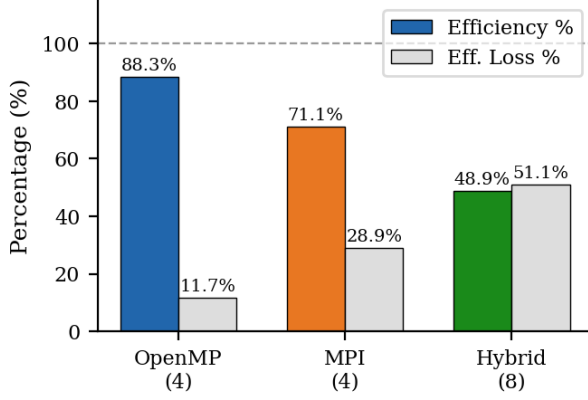


Fig. 2: Parallel Efficiency Analysis

VII. PROFILING ANALYSIS

TAU and mpiP profiling tools were used to analyze execution performance.

A. Communication Overhead

Fig. 4 — Overhead Breakdown (Compute vs I/O)

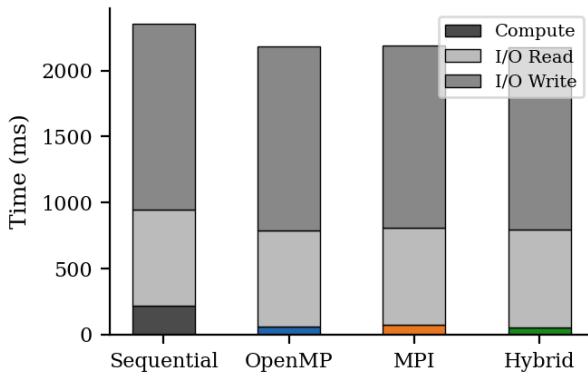


Fig. 3: Communication Overhead Analysis

B. mpiP Profiling

Fig. 5 — mpiP Data Movement Costs (Scatterv & Gatherv)

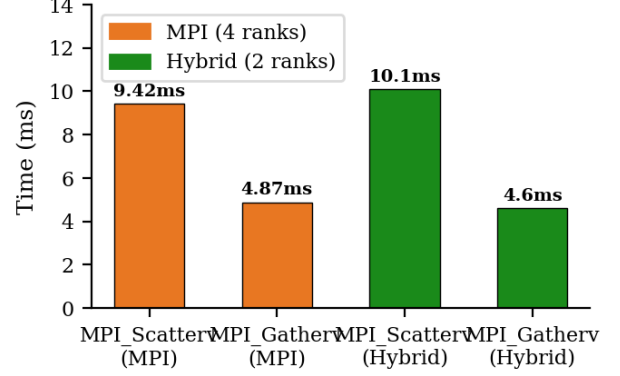


Fig. 4: mpiP Profiling Results

C. Stage Timing Analysis

Fig. 6 — Per-Stage Timing (Sequential vs OpenMP)

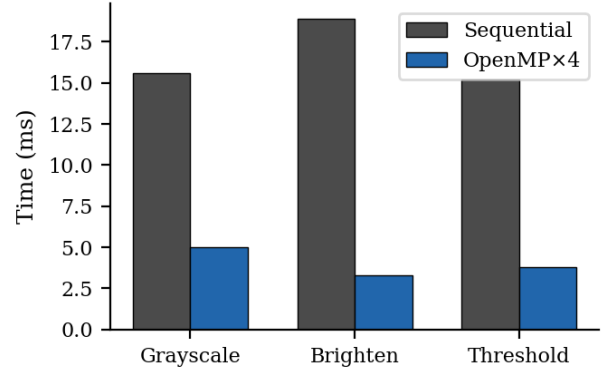


Fig. 5: Per-stage Timing Comparison

VIII. PROFILING RESULTS

TABLE III: TAU Profiling Results

Function	Time(%)	Time(ms)
writePPM	30.4	4561
applyFilters	16.5	2470
readPPM	9.0	1345
MPI_Init	4.0	607
MPI_Gatherv	0.4	66
MPI_Scatterv	0.1	10

IX. CACHE UTILIZATION ANALYSIS

The image working set exceeded available cache size:

$$24.9 \text{ MB} \div 1.25 \text{ MB} \approx 20\times$$

This caused frequent cache misses and memory bandwidth limitations.

- Pipeline became memory-bound
- Threads competed for memory bandwidth
- Efficiency decreased at higher worker counts

X. CORRECTNESS VALIDATION

All parallel outputs matched the sequential baseline.

TABLE IV: Correctness Validation

Implementation	Validation
Sequential	Baseline
OpenMP	Match
MPI	Match
Hybrid	Match

XI. CONCLUSION

This project demonstrated the effectiveness of parallel computing techniques for large-scale image processing on HPC systems.

The Hybrid MPI+OpenMP implementation achieved the best performance with a speedup of 3.91 $\times$ compared to the sequential version.

OpenMP showed better efficiency than MPI because shared memory communication avoids message-passing overhead. Profiling results indicated that the application was primarily limited by I/O and memory bandwidth.

Future improvements include:

- Parallel HDF5 I/O
- GPU acceleration using CUDA
- Dynamic load balancing
- Advanced image filtering techniques

ACKNOWLEDGMENT

The authors thank Texas State University and TACC for access to the Lonestar6 HPC cluster used for project execution and performance analysis.

REFERENCES

- [1] OpenMP Architecture Review Board, “OpenMP Application Program Interface,” Version 5.0, 2018.
- [2] Message Passing Interface Forum, “MPI: A Message Passing Interface Standard,” Version 4.0, 2021.
- [3] Texas Advanced Computing Center, “Lonestar6 User Guide.”
- [4] R. Gonzalez and R. Woods, “Digital Image Processing,” Pearson Education, 2018.
- [5] M. Quinn, “Parallel Programming in C with MPI and OpenMP,” McGraw-Hill, 2003.